PASSO 3: DEPLOY NO RENDER - GUIA COMPLETO E EXPANDIDO (VERSAO 15 PAGINAS)

Como usar este guia

Este documento foi escrito em formato extenso para servir como manual operacional. A proposta e que voce consiga realizar o deploy, validar, corrigir erros, preparar seguranca minima e entrar em rotina de operacao sem depender de tentativa e erro.

A leitura pode ser feita de duas formas:

1. Leitura integral, do inicio ao fim, para quem vai subir o projeto pela primeira vez.
2. Leitura por secoes, para quem ja tem o ambiente funcionando e quer resolver um problema especifico.

Ao longo do texto, voce encontrara:

- explicacoes de contexto (o por que de cada etapa);
- instrucoes praticas (o que clicar e o que preencher);
- sinais de validacao (como saber que deu certo);
- diagnostico de falhas (como corrigir quando algo quebra);
- procedimentos de operacao diaria (para evitar incidentes em producao).

Ilustracoes do projeto

As imagens abaixo ajudam a contextualizar o visual e o tipo de interface que sera publicada.

ObservaTrânsito

---

1. Objetivo real do Passo 3

O Passo 3 nao e apenas "colocar no ar". O objetivo correto e publicar um servico web funcional, com disponibilidade consistente e fluxo de atualizacao previsivel.

Em outras palavras: o app precisa abrir para qualquer usuario com o link publico, registrar dados, manter historico, e continuar operacional apos reinicios, novos deploys e momentos de instabilidade.

Este guia considera um cenario real de uso, em que o sistema de sinistros precisa ficar acessivel para equipe de campo, supervisao e consulta historica.

---

2. Panorama tecnico do que sera publicado

A aplicacao atual usa Flask no backend, interface web no frontend e arquivos locais para persistencia de dados, com suporte a PWA.

No Render, a estrategia recomendada e:

- Build command: instalacao das dependencias Python via requirements.
- Start command: subida do servico via Gunicorn apontando para `server:app`.
- Persistencia: disco montado em `/var/data` para nao perder dados.
- Configuracao de ambiente: chaves de administrador e opcoes de backup.

Sem a parte de persistencia em disco, voce ate consegue abrir o app, mas corre risco de perder registros em reinicio da instancia. Por isso, a configuracao de dados nao e opcional em cenario real.

---

3. Pre-requisitos obrigatorios antes do primeiro clique

Antes de abrir o Render, confirme os itens abaixo.

3.1 Repositorio atualizado

Garanta que o branch principal (`main`) contem os arquivos necessarios:

- `server.py`
- `requirements.txt`
- `Procfile` (se aplicavel ao fluxo)
- pasta `web/` com `index_simple.html`, `manifest.json` e `sw.js`

3.2 Conta GitHub com acesso ao repositorio

Voce precisa ter permissao de leitura do repositorio que sera conectado. Se a conta estiver sem acesso, o repositorio nem aparece na lista do Render.

3.3 Conta Render ativa

Conta gratuita funciona para comecar. No entanto, saiba que plano free pode hibernar e ter "cold start". Mais adiante no guia mostramos como reduzir impacto disso na experiencia.

3.4 Variaveis sensiveis preparadas

Tenha em maos:

- uma chave forte para `ADMIN_ACCESS_KEY`;
- token do GitHub para backup privado (se voce for usar backup automatico).

3.5 Definicao de politica de dados

Defina de antemao quem pode:

- apenas registrar sinistro;
- baixar relatorios;
- disparar backup privado.

Sem essa definicao, e comum expor funcionalidades administrativas para usuarios comuns.

---

4. Criacao da conta e conexao com GitHub no Render

Acesse `https://render.com` e realize login. A opcao mais rapida e autenticar com GitHub, pois reduz etapas de autorizacao na hora de importar repositorio.

Ao autorizar, confira se o Render recebeu permissao para o repositorio correto. Em muitos casos, o usuario libera apenas parte dos repositorios e depois nao encontra o projeto na tela de criacao de servico.

Se isso acontecer:

1. Volte ao GitHub (configuracao de aplicativos instalados).
2. Abra configuracao do Render.
3. Amplie permissao para o repositorio `acidentes-pe`.
4. Retorne ao Render e use refresh na lista.

---

5. Criacao do Web Service (configuracao base)

No dashboard do Render:

1. Clique em `New +`.
2. Selecione `Web Service`.
3. Escolha `Build and deploy from a Git repository`.
4. Selecione o repositorio `acidentes-pe`.

Agora preencha os campos.

5.1 Nome do servico

Escolha um nome sem espacos e sem caracteres especiais. Exemplo:

`observatrafego`

Esse nome influencia a URL final publica.

5.2 Branch

Use `main`, salvo se voce usa outra branch de producao.

5.3 Runtime

Selecione Python.

5.4 Build command

Use exatamente:

```
pip install -r requirements.txt
```

5.5 Start command

Use:

```
python -m gunicorn server:app --bind 0.0.0.0:$PORT
```

Esse formato evita ambiguidades de executavel e tende a ser mais robusto no ambiente gerenciado.

5.6 Plano

Para comecar: `Free`.

5.7 Health check

Se estiver disponivel no painel, configure `healthCheckPath` para:

`/health`

Assim voce monitora rapidamente se a aplicacao esta de pe.

---

6. Configuracao de persistencia (parte critica)

Sem persistencia, deploy pode funcionar e parecer correto no teste inicial, mas registros podem sumir apos restart.

6.1 Criar disco persistente

No servico do Render, adicione um Disk:

- Mount path: `/var/data`
- Tamanho inicial: conforme plano e volume esperado

6.2 Variavel de ambiente para dados

Adicione:

- `DATA_DIR=/var/data`

6.3 O que deve ficar persistente

No seu app, os dados de sinistro e exports devem gravar no caminho persistente, por exemplo:

- `accidents.json`
- pasta `exports/`

Se esses arquivos estiverem em area temporaria da instancia, voce perde historico.

---

7. Seguranca basica para ambiente de producao

7.1 Chave administrativa

Configure:

- `ADMIN_ACCESS_KEY=uma_chave_forte`

Use uma chave longa, sem padrao simples. Evite datas, nomes proprios e sequencias previsiveis.

7.2 Restricao de funcoes sensiveis

Funcoes como download de relatorios e backup devem exigir autenticacao administrativa.

7.3 Token de backup privado (opcional)

Se usar backup no GitHub, configure token com escopo minimo necessario. Nao use token com permissao ampla se nao houver necessidade.

7.4 Boas praticas operacionais

- nunca compartilhar chave administrativa em chats abertos;
- girar a chave periodicamente;
- remover acesso de operadores que nao precisam de perfil admin.

---

8. Configuracao de backup privado no GitHub (opcional, recomendado)

Se voce deseja redundancia de dados, configure variaveis:

- `GITHUB_BACKUP_REPO=seu-user/seu-repo-privado`
- `GITHUB_BACKUP_TOKEN=token_com_permissao_repo`
- `GITHUB_BACKUP_BRANCH=main`
- `GITHUB_BACKUP_PATH=observa_backup`

Com isso, o sistema pode gerar copia externa dos registros sem depender apenas do disco local do Render.

Ponto de atencao: mantenha o repositorio de backup privado e restrinja acesso apenas ao grupo responsavel por governanca de dados.

---

9. Primeiro deploy: o que observar nos logs

Ao clicar em `Create Web Service`, acompanhe logs em tempo real.

Fluxo esperado:

1. Download do codigo.
2. Instalacao de dependencias.
3. Inicializacao do Gunicorn.
4. Disponibilizacao da URL publica.

Sinais positivos:

- build concluido sem erro;
- processo web em estado running;
- endpoint `/health` respondendo 200.

Sinais de problema:

- erro de modulo ausente;
- falha de bind na porta;
- excecao na inicializacao da aplicacao.

---

10. Validacao funcional apos deploy (checklist completo)

Apos ficar `Live`, valide por camadas.

10.1 Disponibilidade

- Abra URL principal no navegador desktop.
- Abra URL no celular.
- Teste `/health`.

10.2 Formulario de sinistro

- Preencha instituicao/notificante/endereco.
- Marque coordenadas no mapa.
- Envie descricao.

10.3 Fotos

Com as abas recentes da interface, valide os dois fluxos:

1. `Selecionar fotos da galeria`
2. `Tirar fotos da camera`

Verifique se preview aparece e se limite de quantidade e respeitado.

10.4 Offline e sincronizacao

- Simule perda de internet.
- Grave registro offline.
- Volte online e sincronize.

10.5 Tempo de abertura

- Feche e reabra app.
- Observe renderizacao inicial.
- Verifique se lista em cache aparece rapido.

10.6 Exportacoes (admin)

Apos login administrativo:

- baixar planilha diaria;
- baixar mapa diario;
- validar se arquivo abre corretamente.

---

11. Rotina de deploy continuo (push para producao)

Depois do primeiro deploy, atualizacoes seguem fluxo padrao:

1. Desenvolver localmente.
2. Rodar teste basico.
3. Commit com mensagem clara.
4. Push para `main`.
5. Acompanhar redeploy automatico no Render.

Padrao de mensagem recomendado:

- `feat:` para funcionalidade nova;
- `fix:` para correcao;
- `perf:` para performance;
- `docs:` para documentacao.

Esse padrao facilita rastreabilidade e auditoria de mudancas.

---

12. Troubleshooting aprofundado

12.1 Erro: "Application failed to start"

Possiveis causas:

- comando start incorreto;
- modulo principal errado;
- dependencia faltando no `requirements.txt`.

Acao:

1. conferir start command;
2. validar import principal no `server.py`;
3. revisar dependencias;
4. fazer novo deploy.

12.2 Erro 404 na raiz

Possiveis causas:

- pasta `web/` ausente no build;
- arquivo principal diferente do esperado;
- rota raiz sem fallback.

Acao:

1. verificar existencia de `web/index_simple.html`;
2. confirmar logica da rota `/` no backend;
3. redeploy apos commit.

12.3 Geolocalizacao indisponivel

Possiveis causas:

- permissao negada no navegador;
- dispositivo sem servico de localizacao ativo;
- uso de navegador com restricoes especificas.

Acao:

1. liberar localizacao no navegador;
2. testar em Safari (iPhone) ou Chrome (Android);
3. confirmar HTTPS.

12.4 Upload de fotos falhando

Possiveis causas:

- fotos acima do limite pratico;
- conexao instavel;
- permissao de camera/galeria negada.

Acao:

1. testar foto menor;
2. verificar permissao do sistema;
3. tentar novamente em rede estavel.

12.5 Lentidao na abertura

Possiveis causas:

- cold start do plano free;
- rede movel fraca;
- volume alto de registros e imagens.

Mitigacoes:

- manter cache funcional;
- evitar assets pesados desnecessarios;
- considerar upgrade de plano para uso intenso.

---

13. Procedimento de rollback seguro

Se uma versao nova causar problema em producao, execute rollback de forma controlada:

1. identifique ultimo commit estavel;
2. reverta no Git com commit de reversao (evite comando destrutivo);
3. push para `main`;
4. acompanhe redeploy;
5. valide endpoints principais.

Nunca faca rollback "no escuro" sem validar logs e impacto, para nao trocar um problema por outro.

---

14. Operacao diaria (manual de campo)

14.1 Inicio do turno

- abrir sistema e testar carregamento;
- validar conectividade;
- confirmar que registro de teste funciona.

14.2 Durante o turno

- registrar cada sinistro com dados minimos obrigatorios;
- anexar imagens quando disponiveis;
- revisar pendencias offline.

14.3 Fim do turno

- sincronizar registros pendentes;
- conferir exportacao diaria;
- opcionalmente executar backup privado.

---

15. Qualidade de dados e padronizacao de preenchimento

Um sistema de sinistro so gera inteligencia util quando os dados sao consistentes. Para isso, adote orientacoes operacionais:

- endereco do sinistro com referencia clara;
- descricao breve, objetiva e sem ambiguidade;
- instituicao preenchida de forma padrao (sem variacoes desnecessarias);
- fotos com enquadramento util para analise.

Padronizacao reduz retrabalho e aumenta qualidade de relatorio.

---

16. Governanca e conformidade

Mesmo em projeto inicial, trate governanca como requisito.

- mantenha historico de alteracoes no Git;
- registre quem possui chave administrativa;
- defina responsavel por backup e auditoria;
- documente processo de resposta a incidente.

Se o uso crescer, essa base de governanca evita perdas, conflitos e exposicao indevida de informacao.

---

17. Monitoramento minimo recomendado

No minimo, monitore:

- status do servico (Live/Unhealthy);
- tempo de resposta da rota principal;
- erros recorrentes em logs;
- volume diario de registros;
- sucesso/fracasso de sincronizacao offline.

Com esse conjunto, voce identifica degradacao cedo e consegue agir antes do problema virar indisponibilidade prolongada.

---

18. Estrategia de capacidade e crescimento

Quando o numero de usuarios aumentar, alguns limites aparecem:

- maior concorrencia de acesso;
- payload maior por causa de imagens;
- aumento de volume de historico.

Evolucoes naturais:

1. migrar persistencia para banco gerenciado;
2. mover assets de imagem para storage dedicado;
3. separar API e frontend em camadas independentes;
4. introduzir observabilidade estruturada.

Planejar isso cedo evita migracoes emergenciais sob pressao.

---

19. Plano de contingencia

Se o servico principal ficar indisponivel:

1. manter coleta local (offline);
2. comunicar equipe sobre modo contingencia;
3. restaurar servico principal;
4. sincronizar backlog;
5. emitir relatorio de incidente com causa raiz.

A existencia de modo offline e um diferencial importante para continuidade operacional.

---

20. Checklist final de aprovacao do Passo 3

Use esta lista como criterio de pronto.

20.1 Infraestrutura

- [ ] Servico criado no Render
- [ ] Branch correta apontada
- [ ] Build e start command corretos
- [ ] Health check configurado

20.2 Dados e persistencia

- [ ] Disk persistente montado em `/var/data`
- [ ] `DATA_DIR` configurada
- [ ] Registros continuam apos restart

20.3 Seguranca

- [ ] `ADMIN_ACCESS_KEY` configurada
- [ ] Funcoes sensiveis restritas ao admin
- [ ] Token de backup com escopo minimo (se aplicavel)

20.4 Funcionalidade

- [ ] Cadastro de sinistro funcionando
- [ ] Upload por galeria funcionando
- [ ] Captura por camera funcionando
- [ ] Mapa, GPS e cronometro funcionando
- [ ] Offline + sincronizacao funcionando

20.5 Operacao

- [ ] Exportacoes administrativas funcionando
- [ ] Backup privado testado (se aplicavel)
- [ ] Procedimento de rollback definido
- [ ] Rotina diaria documentada para equipe

---

21. Resumo executivo

Se voce chegou ate aqui com todos os itens marcados, seu deploy no Render nao esta apenas "no ar". Ele esta operavel, monitoravel e com base de seguranca e governanca suficientes para uso real.

Esse e o ponto em que o projeto deixa de ser prototipo e passa a operar como servico de verdade.

---

22. Proximos passos recomendados apos o Passo 3

1. Consolidar indicadores (volume diario, localizacao, horario, reincidencia).
2. Padronizar exportacao para consumo por areas gestoras.
3. Criar rotina de revisao semanal com equipe operacional.

4. Avaliar trilha de publicacao nativa para iOS/Android quando necessario.
5. Evoluir controle de acesso para perfis e nao apenas chave unica.

Apendice A - Campos de configuracao (copiar e colar)

Render

- Build command:

```
pip install -r requirements.txt
```

- Start command:

```
python -m gunicorn server:app --bind 0.0.0.0:$PORT
```

- Health check:

```
/health
```

Environment variables

```
DATA_DIR=/var/data
ADMIN_ACCESS_KEY=sua_chave_forte
GITHUB_BACKUP_REPO=seu-user/seu-repo-privado
GITHUB_BACKUP_TOKEN=seu_token
GITHUB_BACKUP_BRANCH=main
GITHUB_BACKUP_PATH=observa_backup
```

Apendice B - Diagnostico rapido em 5 minutos

Se algo parou e voce precisa de resposta rapida:

1. Abra `Logs` no Render.
2. Teste `GET /health`.
3. Teste `GET /api/accidents`.
4. Valide escrita de novo registro.
5. Verifique se arquivo de dados continua no path persistente.

Se os cinco pontos acima estiverem ok, o nucleo da aplicacao esta saudavel.

Encerramento

Este documento foi expandido para funcionar como referencia operacional de longo prazo. Use-o como guia de implantacao, manual de suporte e material de onboarding para novas pessoas da equipe.

Quando houver mudanca relevante na arquitetura ou no fluxo de deploy, atualize este arquivo no mesmo commit da alteracao tecnica. Essa disciplina evita divergencia entre "o que esta escrito" e "o que esta rodando".